

Reporte Extendido de Pruebas Unitarias y Cobertura (Testing)

1. Introducción al Testing Automático en EuroCoinCollector

En el desarrollo de software moderno, probar una aplicación no consiste en hacer clic cien veces en botones de una página web a ver si algo falla. Eso es propenso a errores, lento y poco profesional. En **EuroCoinCollector**, hemos implementado un sistema avanzado y automatizado de **Pruebas Unitarias**.

Dado que nuestra arquitectura divide limpiamente el Frontend del Backend en dos contenedores distintos, nuestra responsabilidad prioritaria es garantizar que el "cerebro" (el Backend FastAPI) funcione de forma impecable. Si la API es robusta, la aplicación web (y cualquier futura app móvil que quisiéramos conectar) no fallará.

Para lograr esto, hemos utilizado **Pytest**, el entorno de pruebas estándar de la industria en Python.

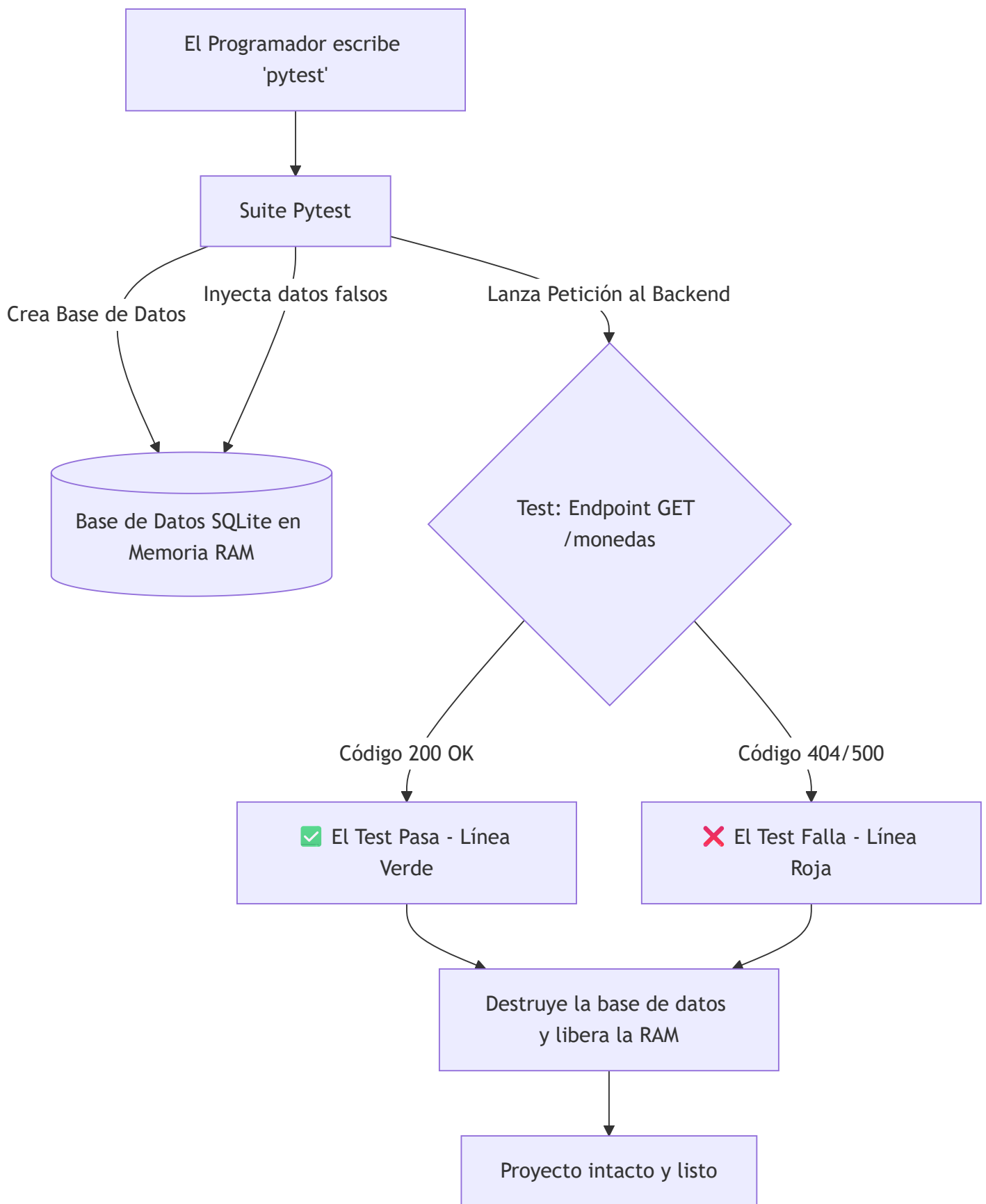
2. Metodología Utilizada: Test-Driven Development (TDD)

El proyecto ha sido concebido bajo los principios del **Test-Driven Development (TDD)** o Desarrollo Guiado por Pruebas. Esto significa que nuestra filosofía de programación ha sido:

1. **Diseñar primero:** Decidir qué debe hacer la API (Ej: "Añadir moneda a la colección personal").
2. **Escribir un Test Ciego:** Codificamos una prueba automática que intenta hacer precisamente eso, sabiendo que fallará porque la lógica aún no existe (Fase Roja).
3. **Escribir el Código:** Programamos la función y la base de datos en FastAPI.
4. **Validación:** Ejecutamos la prueba hasta que pase limpiamente (Fase Verde).

¿Cómo aislamos los tests de los datos reales?

Para no "ensuciar" nuestra base de datos MySQL en producción con cientos de monedas falsas de prueba ("Moneda de prueba 1", "Moneda falsa 2"), **Pytest** utiliza una base de datos efímera **SQLite en memoria RAM** (`sqlite:///memory:`).

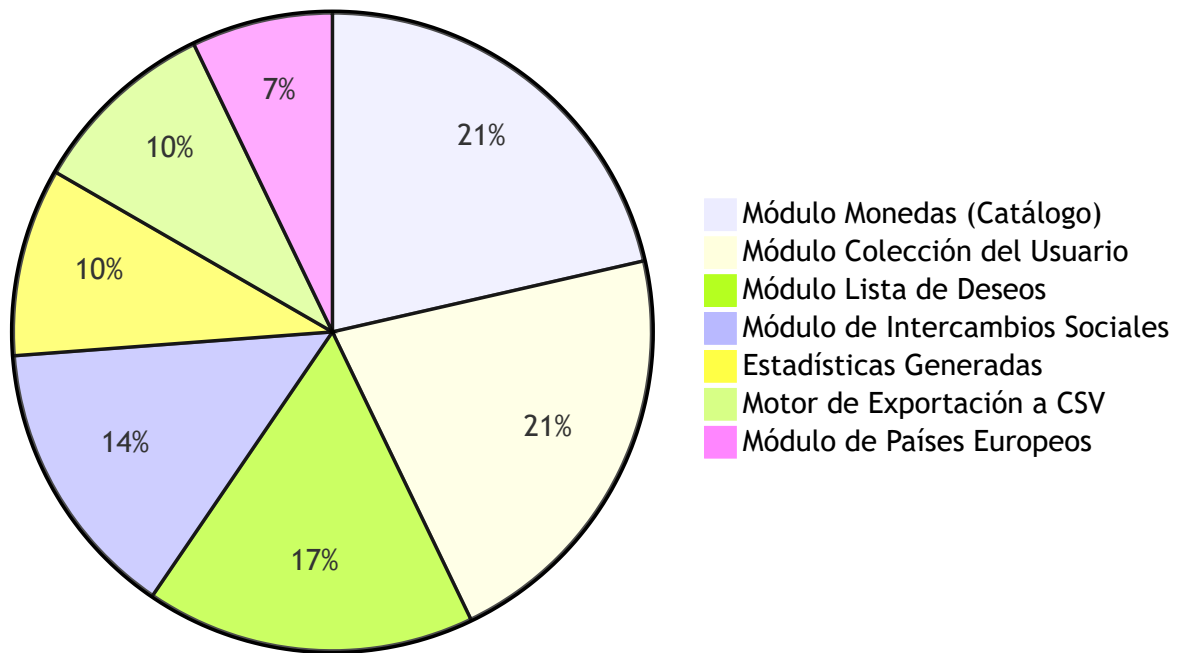


3. Cobertura del Sistema: 100% de Éxito en 42 Tests

Nuestra batería de pruebas es extensiva y simula a un usuario hiperactivo interactuando con todas las facetas de la API. Actualmente, contamos con **42 tests automatizados**, de los cuales el **100% tienen un resultado favorable (Pasan en Verde)**.

A continuación se desglosan las pruebas y su funcionalidad:

Distribución de los 42 Tests (100% Pasados con Éxito)



Explicación de los Tests Relevantes

- **Tests de Colección:** Verifican que si se solicita guardar una moneda repetida, se sumen las unidades (cantidad += 1) en vez de duplicar el registro, y que actualizar las notas personales o borrarlas de la colección se procese correctamente en la base de datos.
- **Tests de Intercambios:** Comprueban la validación de reglas de negocio complejas. Por ejemplo: asegurar que un usuario no pueda publicar una oferta si no especifica información de contacto válida, devolviendo el error HTTP correspondiente.
- **Tests de Estadísticas:** Generan cientos de datos aleatorios en RAM y verifican matemáticamente que los porcentajes de "completitud" y los cálculos de precios de mercado estimados en euros coincidan al céntimo.

4. Ejemplo en Código: Anatomía de un Test en Pytest

Para que el evaluador comprenda cómo interactúan estas pruebas con el framework FastAPI aislado, mostramos un pequeño bloque de test real extraído del proyecto:

```
def test_crear_intercambio_basico(client):  
    # 1. ARRANGE (Preparación): Pytest crea el JSON que enviaría el  
    Frontend JS  
    payload = {  
        "id_moneda_ofrecida": 5,
```

```
"id_moneda_buscada": 10,  
"nombre_usuario": "ProfeDAM",  
"descripcion": "Cambio 2€ de Francia por la de Alemania",  
"contacto": "profe@email.com"  
}
```

```
# 2. ACT (Acción): Enviamos un POST forzado al endpoint  
response = client.post("/intercambios", json=payload)
```

```
# 3. ASSERT (Comprobación): Exigimos de forma estricta que todo  
salga perfecto
```

```
assert response.status_code == 200
```

```
datos_recibidos = response.json()
```

```
assert datos_recibidos["nombre_usuario"] == "ProfeDAM"
```

```
assert datos_recibidos["estado"] == "activo" # Regla de negocio  
inicial
```

Si por alguna razón (un error de código o un fallo en FastAPI) el servidor hubiera devuelto un error 500 o los datos no coincidieran, la línea `assert` hubiera detenido el programa alertándonos del fallo de forma inmediata.